

LSTM-Guided Brute-Force: An AI-Enhanced Approach for the ESC 2025 GateKeeper’s Revenge Qualification Challenge

Dhairya Gupta
Dept. of Electrical Engineering
IIT Kanpur
dhairvag24@iitk.ac.in

Dhruv Garg
Dept. of Aerospace Engineering
IIT Kanpur
dhruvg24@iitk.ac.in

Pratham Gupta
Dept. of Electrical Engineering
IIT Kanpur
prathamg24@iitk.ac.in

Aritra Kolay
Dept. of Electrical Engineering
IIT Kanpur
aritrak23@iitk.ac.in

Shubham Sahay
Associate Professor
Dept. of Electrical Engineering
IIT Kanpur
ssahay@iitk.ac.in

Abstract—This report describes an LSTM-guided enhancement to brute-force search for the ESC 2025 “GateKeeper’s Revenge” qualification. The provided `gatekeeper.c` routine leaks secret-dependent timing and power via an early-exit comparison and a conditional busy-wait. After a focused analysis of that vulnerability, we explain why naive brute force is impractical, briefly review standard side-channel techniques that motivate measurement choices, and present our proposed approach: a pipeline that uses ROI discovery, a CNN→LSTM probabilistic classifier to produce per-byte posteriors, and beam/A*-style search that brute-forces only the most promising candidate tails. We provide model design, hybrid-search pseudocode, evaluation metrics, and mitigations.

I. VULNERABILITY: ANALYSIS OF GATEKEEPER.C

The supplied source implements a password verification routine that compares an input array data against a static password byte-by-byte. Key behavior (conceptual):

```
for (uint8_t i = 0; i < password_len; i++) {
    if (data[i] == password[i]) {
        matched_chars++;
        for (volatile uint32_t j = 0; j < 5000; j++);
    } else {
        break;
    }
}
```

Additional observations:

- **Early exit:** the loop breaks on the first mismatch, so execution time depends on the length of the correct prefix.
- **Busy-wait per match:** matched iterations execute a long busy-wait, producing a sustained, observable power/timing window.
- **Trigger pins:** the code toggles trigger signals (`trigger_high()`/`trigger_low()`) enabling precise alignment of captured traces.

These features produce two exploitable side channels: (1) timing differences correlated to the number of correct leading bytes, and (2) per-iteration power signatures (or EM) that differ between matched and mismatched iterations. The combination allows an attacker to infer the password incrementally instead of searching the full space.

II. BRUTE-FORCE: INTRODUCTION AND LIMITATIONS

Brute force enumerates every candidate in the keyspace. If password length is L and alphabet size is A , the search space is A^L .

A. Complexity illustration

Examples:

- $A = 95$ (printable ASCII), $L = 5$: $95^5 \approx 7.7 \times 10^9$ candidates.
- $A = 256$ (full byte), $L = 5$: $256^5 \approx 1.1 \times 10^{12}$ candidates.

Interactive device constraints (UART latency, MCU processing, I/O overhead) typically limit attempts to 10^1 – 10^4 /s. At realistic rates brute force is impractical on live hardware; rate-limiting or lockout further frustrates it.

B. Why side-channels change the game

Side-channels can convert an A^L problem into a sequence of much smaller tasks (e.g., per-byte recovery). If the attacker can determine the i -th byte independently, worst-case work becomes $L \times A$. Better yet, a model that predicts a small top- k set per byte reduces combinatorics to $\prod_i k_i$ (often tiny compared to A^L). Our approach exploits this property.

III. STANDARD SIDE-CHANNEL TECHNIQUES (SHORT PRIMER)

This brief primer explains standard techniques motivating measurements used in our pipeline.

A. Simple Power Analysis (SPA)

SPA inspects individual traces visually to find distinctive patterns (spikes, loops). The busy-wait region in matched cases is often visible in SPA.

B. Differential Power Analysis (DPA)

DPA partitions traces by a hypothesis (e.g., guessed value) and computes mean differences between groups; a significant peak suggests a correct hypothesis [11].

C. Correlation Power Analysis (CPA)

CPA computes correlation between a leakage model (e.g., Hamming weight) and measured samples; the hypothesis with maximal correlation is favored [12].

D. Timing analysis

Measure elapsed time per invocation (trigger-to-trigger). Early-exit patterns produce timing distributions correlated with correct prefix length [10].

E. Trace alignment and ROI selection

High-quality SCA requires aligned traces and selection of Regions of Interest (ROIs). The device triggers enable reliable alignment; spectral/CNN detectors can help find ROIs robustly when jitter exists.

IV. PROPOSED APPROACH: LSTM-GUIDED BRUTE-FORCE

We develop a pipeline that uses an LSTM-based classifier to guide a hybrid brute-force search. The core idea: infer per-position likelihoods from traces, then search the small, high-probability subspace first.

A. Overall pipeline

- 1) **Data acquisition & alignment:** capture many traces with known plaintext inputs; align via triggers.
- 2) **ROI discovery & preprocessing:** crop trace windows containing the comparison/busy-wait; denoise and normalize.
- 3) **LSTM-based probabilistic classification:** train a CNN→LSTM model to output $P_i(b \mid \text{trace})$, a posterior over bytes b for position i .
- 4) **Hybrid search:** fuse per-position posteriors and optional LM priors; perform beam/A* search and locally brute-force low-depth tails.
- 5) **Active probing (optional):** pick new inputs that maximize expected information gain among top hypotheses, capture new traces, and refine posteriors.

B. Preprocessing & ROI extraction

Recommended steps:

- Align traces using trigger transitions.
- Crop a window (e.g., 200–800 samples) around the compare/busy-wait region.
- Apply bandpass filtering or baseline removal.
- Normalize per-trace (z-score).
- Augment (small shifts, additive noise, amplitude scaling) for robustness.

C. Model architecture and training

A practical starter model:

- **CNN front end:** Conv1D(16, k=11) → ReLU → MaxPool; Conv1D(32, k=7) → ReLU → MaxPool.
- **Temporal layer:** Bidirectional LSTM(128) with dropout.
- **Output:** Dense(256) → Softmax (or Dense($|A|$) for smaller alphabets).

Training:

- Loss: categorical cross-entropy.
- Optimizer: Adam with learning-rate scheduling.
- Regularization: dropout, early stopping.
- Calibration: temperature scaling to produce reliable probabilities.
- Uncertainty: estimate via MC dropout or model ensembles; high entropy indicates low confidence.

D. Combining model outputs with search

For a candidate $c = (b_1, \dots, b_L)$ use a log-score:

$$\log \text{score}(c) = \sum_{i=1}^L \alpha \log P_i(b_i) + \beta \log P_{\text{LM}}(c),$$

where P_{LM} is an optional language/format prior and α, β weight evidence vs prior.

E. Hybrid search algorithms

We present a practical beam + local brute-force algorithm (see Algorithm 1) and an A* variant for best-first exploration.

Algorithm 1 BeamSearchWithTailBF

Require: L , per-pos posteriors $P_i(b)$, beam size B , topK K , tail limit T

```

1: beam  $\leftarrow [("", \text{score}=0)]$ 
2: for  $i = 1$  to  $L$  do
3:   candidates  $\leftarrow \{\}$ 
4:   for (pref,score) in beam do
5:     for  $b$  in topK( $P_i, K$ ) do
6:       new_pref  $\leftarrow \text{pref} \parallel b$ 
7:       new_score  $\leftarrow \text{score} + \log P_i(b)$ 
8:       candidates.add(new_pref,new_score)
9:   end for
10:  end for
11:  beam  $\leftarrow \text{topB}(\text{candidates})$ 
12: end for
13: results  $\leftarrow []$ 
14: for (pref,score) in beam do
15:   if len(pref)== $L$  then results.add(pref,score)
16:   else
17:     brute-force tails up to  $T$  attempts, ranked by
       posterior, add full candidates to results
18:   end if
19: end for
20: return sort(results)

```

The A* variant uses admissible heuristic $h(\text{prefix}) = -\sum_{j=\ell+1}^L \max_b \log P_j(b)$ and expands prefixes with smallest $f = g + h$.

F. Active probing

When top candidates tie or posterior entropy is high, select inputs that maximize expected information gain among top hypotheses, collect new traces, and update posteriors. This reduces the number of device attempts needed.

V. EVALUATION PLAN AND METRICS

Follow SCA evaluation best-practices:

- **Data splits:** use separate capture sessions for train/val/test.
- **Metrics:** Top- k accuracy (per position), Guessing Entropy (GE), Traces-to-Success (TTS), Candidates-to-Success, and cross-session generalization.
- **Ablation:** evaluate spectral vs raw input, CNN vs LSTM vs hybrid, beam size, top- K , and LM prior weight.
- **Statistics:** report mean and standard deviation over multiple seeds/splits.

VI. PRACTICAL PARAMETER RECOMMENDATIONS

Initial parameters to try:

- Beam $B = 50$, topK $K = 10$, tail brute-force cap $T = 1000$.
- If no success within top M (e.g., 500), increase K or request more traces.
- For low-data regimes: pretrain on public SCA datasets (ASCAD) and fine-tune.

VII. MITIGATIONS

To eliminate the leakage targeted by this approach:

- **Constant-time compare:** avoid early-exit, perform full-length comparison and reduce branch dependence.
- **Dummy operations/masking:** ensure per-iteration work is independent of matches.
- **Rate-limiting and anomaly detection:** enforce attempt limits and flag suspicious probing patterns.
- **Hardware protections:** brown-out/glitch detectors and watchdog timers.

VIII. RELATED WORK

This approach builds on deep-learning SCA research: Maghrebi et al. (CNNs for SCA) [1]; Benadjila et al. (ASCAD dataset) [2]; Kim et al. (CNN architectures augmentation) [3]; Ahmed & Hasan (LSTM for SCA) [4]; SoK and evaluation

guidance by Picek et al. and Wu [5], [6]. Detection and attention-based preprocessing techniques inspire robust ROI selection [7]. LLM-assisted automation has been recently explored in practice [8]. Practical instrumentation references include ChipWhisperer documentation [9].

IX. CONCLUSION

We presented an LSTM-guided enhancement to brute-force for the GateKeeper challenge: align traces and extract ROIs, train a CNN→LSTM to produce per-byte posteriors, and run a beam/A*-style hybrid search that brute-forces only short candidate tails. This dramatically reduces practical attempts compared to exhaustive search while retaining completeness through controlled fallback. The proposed pipeline follows SCA evaluation best-practices and includes uncertainty handling and active probing to minimize device interactions.

REFERENCES

- [1] H. Maghrebi, T. Portigliatti, and E. Prouff, “Breaking cryptographic implementations using deep learning techniques,” in *CHES Workshops (LNCS)*, 2016.
- [2] R. Benadjila et al., “Deep learning for side-channel analysis and introduction to the ASCAD database,” *Journal of Cryptographic Engineering*, 2018.
- [3] J. Kim et al., “Make some noise: Convolutional neural networks for profiled side-channel analysis,” IACR/CHES workshop, 2019.
- [4] A. A. Ahmed and M. K. Hasan, “Design and implementation of side channel attack based on deep learning LSTM,” TENSYP, 2023.
- [5] S. Picek et al., “SoK: Deep learning-based physical side-channel analysis,” *ACM Computing Surveys*, 2023.
- [6] L. Wu, “On the evaluation of deep learning-based side-channel analysis,” 2022.
- [7] S. Narla, “Security-aware side-channel detection through convolutional transformer networks and hybrid LSTM-spectral analysis,” 2024.
- [8] W. Waligóra, “Large language models in side-channel cryptanalysis,” 2025 (preprint).
- [9] C. O’Flynn, “Fault injection and the ChipWhisperer platform,” ChipWhisperer docs, 2019.
- [10] P. Kocher, “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems,” in *CRYPTO*, 1996.
- [11] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *CRYPTO*, 1999.
- [12] W. Schindler, “A software-based power analysis,” *Journal of Cryptographic Engineering*, 2005.